

Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Eléctrica

**Sistema de Monitoreo Remoto
para Inversor Solar Riello
H.P.6065REL-D**

Informe Técnico

Daniel Ruminot Moscoso

17 de junio de 2026

Concepción, Chile

Índice

1. Resumen Ejecutivo	3
2. Introducción	3
2.1. Motivación	3
2.2. Alcance	4
2.3. Objetivos	4
3. Marco Teórico	5
3.1. Inversor Solar Riello H.P.6065REL-D	5
3.2. Protocolo SISER (Phoenixtec)	5
3.2.1. Estructura del frame SISER	5
3.2.2. Secuencia de handshake	6
3.2.3. Comando readMichele (datos del inversor)	6
3.2.4. Control de flujo serial	7
3.2.5. Configuración serial	8
3.2.6. Adaptadores de Comunicación Soportados	8
3.3. Protocolo Modbus RTU (exploración inicial)	8
3.4. Software SunVision y Protocolo NetMan UDP	8
3.4.1. SunVision en Docker via Wine (eliminado)	9
3.4.2. Protocolo NetMan UDP	9
3.5. Base de Datos de Series Temporales: TimescaleDB	9
3.6. Acceso Remoto: ngrok + nginx Reverse Proxy	10
3.6.1. ngrok	10
3.6.2. nginx	10
3.6.3. Chrome Remote Desktop	10
4. Arquitectura del Sistema	11
4.1. Diagrama General	11
4.2. Stack de Software	12
4.3. Servicios Docker	12
4.4. Rutas del Reverse Proxy (nginx)	12
4.5. Limitaciones de ngrok Free	12
5. Implementación	13
5.1. Daemon SISER Reader (Python)	13
5.1.1. Características Clave	13
5.1.2. Datos leídos del inversor	14
5.2. Daemon Modbus Reader (C) — Legacy	15
5.3. Esquema de Base de Datos	15
5.4. Dashboards Grafana	15
5.5. Simulador del Inversor (Modbus RTU)	16
5.5.1. Modos del Simulador	16
5.6. SunVision en Docker via Wine (eliminado)	16
5.7. Terminal Web (ttyd)	17
5.8. Servidor de Comandos Remotos (cmd-server.py)	17
5.9. Pantalla Kiosco	17

5.10. Acceso Remoto	18
5.10.1. ngrok + nginx (Acceso Principal)	18
5.10.2. Chrome Remote Desktop (Acceso GUI)	19
5.10.3. Redes del servidor lautaro	19
6. Reverse Engineering del Mapa de Registros	19
6.1. Contexto y Estado Actual	19
6.2. Método de Descubrimiento	19
6.2.1. Decompilación de SunVision (método principal)	19
6.2.2. Análisis del protocolo NetMan UDP	20
6.2.3. Escaneo sistemático con pymodbus	20
6.3. Registros Implementados y Estado Operativo	21
6.4. Flujo Operativo del Sistema	22
6.5. Confirmación con Inversor Real	23
6.6. Variables Objetivo	24
7. Testing y Verificación	24
7.1. Categorías de Testing	24
7.2. Métricas de Rendimiento	25
8. Despliegue y Operación	27
8.1. Servicios en Producción	27
8.2. Logros Destacados	29
8.3. Trabajo Futuro	29
9. Riesgos y Mitigaciones	29
10. Conclusiones	29
11. Referencias	30

1 Resumen Ejecutivo

Este informe documenta el diseño, implementación y despliegue de un sistema de monitoreo remoto para el inversor solar Riello **H.P.6065REL-D** instalado en el laboratorio de la Universidad de Concepción. El sistema reemplaza el software propietario SunVision, que presenta inestabilidad y requiere Windows, por una solución basada en software libre, accesible desde cualquier navegador web.

La arquitectura utiliza un daemon en Python (**siser-reader**) que se comunica con el inversor vía el protocolo binario SISER (Phoenixtec) sobre RS232 (adaptador CH340 USB-RS232), TimescaleDB para almacenamiento de series temporales, Grafana para visualización mediante consultas SQL directas, y ngrok + nginx para acceso remoto seguro sin necesidad de VPN ni puertos abiertos.

El protocolo SISER fue descubierto mediante decompilación de SunVision y permite leer datos de los 3 MPPTs del inversor trifásico (voltaje, corriente y potencia por cada MPPT), temperatura, voltaje y corriente de red, frecuencia, estado, y energía acumulada. El sistema está **leyendo datos reales del inversor** desde junio 2026, con 22.000+ registros en TimescaleDB.

Se implementaron adicionalmente un **simulador multifuncional** (Modbus RTU, NetMan UDP y SISER) para testing sin el inversor físico, una **pantalla kiosco** para visualización local en el laboratorio, una **terminal web** (ttyd) y un **servidor de comandos remotos** (cmd-server) para administración. SunVision fue ejecutado temporalmente en Docker via Wine para el reverse engineering del protocolo SISER, pero fue eliminado del stack de producción tras completar el descubrimiento del protocolo.

El sistema está diseñado para soportar múltiples usuarios simultáneos (estudiantes y académicos), con acceso público al dashboard de Grafana y autenticación Basic para funciones de administración.

2 Introducción

2.1 Motivación

El inversor solar Riello H.P.6065REL-D instalado en la Universidad de Concepción actualmente se monitorea mediante el software propietario SunVision v1.9.3, que presenta los siguientes problemas:

- Funciona únicamente en Windows (inestable en Linux nativo)
- Crasheos frecuentes al conectar/desconectar el adaptador USB-RS232
- Requiere conexión física al inversor
- No permite acceso remoto desde múltiples dispositivos
- No ofrece API ni integración con otros sistemas
- Interfaz Java Swing obsoleta y pesada
- Exportación limitada a capturas de pantalla

Se requiere un sistema de monitoreo que:

- Sea accesible desde cualquier navegador web
- Funcione en Linux (Ubuntu Server)
- Permita acceso simultáneo de múltiples usuarios
- Permita exportación de datos (CSV, JSON)
- Sea robusto ante desconexiones del adaptador USB-RS232

2.2 Alcance

Este proyecto cubre:

- Comunicación Modbus RTU con el inversor vía RS232 (adaptador PL2303 USB-RS232)
- Protocolo de unlock del inversor Riello (secuencia de autenticación Modbus)
- Almacenamiento de datos en serie temporal con resolución estratificada
- Dashboards interactivos en tiempo real e históricos (Grafana con SQL directo)
- Simulador Modbus RTU para pruebas sin inversor físico
- Acceso remoto seguro sin VPN (ngrok + nginx reverse proxy)
- Terminal web (ttyd) y servidor de comandos remotos (cmd-server.py)
- SunVision fue usado temporalmente para reverse engineering del protocolo SISER (eliminado del stack de producción)
- Pantalla kiosco en modo 24/7 para visualización local en el laboratorio
- Suite de testing automatizado (SQL, operación)
- Documentación técnica y tutorial de uso para estudiantes

2.3 Objetivos

Objetivo General: Diseñar e implementar un sistema de monitoreo remoto robusto para el inversor solar Riello H.P.6065REL-D, accesible desde cualquier navegador web.

Objetivos Específicos:

1. Establecer comunicación confiable con el inversor vía protocolo SISER (Phoenixtec) sobre RS232 (9600 bps, dirección 33)
2. Implementar handshake de registro (offlineEnquiry + sendAddress) antes de cada sesión de lectura
3. Implementar reconexión automática ante desconexiones USB y fallos de handshake
4. Almacenar datos históricos con compresión y retención configurables (TimescaleDB)
5. Crear dashboards académicamente relevantes (potencia, energía, eficiencia, CO2)

6. Configurar acceso remoto seguro mediante ngrok + nginx reverse proxy
7. Implementar herramientas de administración remota (ttyd, cmd-server)
8. Implementar pantalla kiosco 24/7 para el laboratorio

3 Marco Teórico

3.1 Inversor Solar Riello H.P.6065REL-D

El inversor instalado en la Universidad de Concepción es el modelo **H.P.6065REL-D** de Riello, un inversor trifásico con 3 MPPTs, con las siguientes características:

Parámetro	Valor
Potencia nominal	6 kW
Tipo	Trifásico (3 MPPTs)
Conexión	RS232 (DB15/VGA)
Protocolo	SISER (Phoenixtec binario)
Baud rate	9600 bps
Dirección	33 (asignada por handshake)
Paridad	8N1 (sin paridad)

Cuadro 1: Parámetros del inversor H.P.6065REL-D

Importante

El inversor H.P.6065REL-D utiliza el protocolo binario propietario SISER (Phoenixtec), no Modbus RTU. El protocolo SISER requiere un handshake de registro (offlineEnquiry + sendAddress) antes de poder leer datos, y usa control de flujo RTS/DTR para la comunicación con el optoacoplador interno. El daemon `siser-reader` implementa este handshake automáticamente.

3.2 Protocolo SISER (Phoenixtec)

El inversor Riello H.P.6065REL-D utiliza el protocolo binario propietario **SISER** (Phoenixtec), descubierto mediante decompilación de SunVision v1.9.3 (clase `SISERBus.java`). Este protocolo es completamente distinto de Modbus RTU y se comunica vía RS232 con control de flujo RTS/DTR para el optoacoplador interno del inversor.

Inicialmente se exploró el protocolo Modbus RTU (FC03 Read Holding Registers, FC10 Write Multiple Registers), pero el inversor no respondió de forma consistente a Modbus. La decompilación reveló que SunVision utiliza el protocolo SISER en su lugar, con un handshake de registro obligatorio antes de cada sesión de lectura.

3.2.1. Estructura del frame SISER

El frame SISER tiene la siguiente estructura:

Campo	Tamaño	Descripción
Sync	2 bytes	0xAA 0xAA (bytes de sincronización)
Reserved	2 bytes	0x01 0x00
Address	1 byte	Dirección del dispositivo (0x00 para of- line)
Group	1 byte	Grupo de comando (0=registro, 1=da- tos)
Command	1 byte	Código de coman- do (0x00=offlineEnquiry, 0x10=readMichele, etc.)
Data Length	1 byte	Longitud de los datos (0-255 bytes)
Data	N bytes	Datos específicos del comando
Checksum	2 bytes	Suma aditiva de todos los bytes excepto los últimos 2, big-endian

Cuadro 2: Estructura del frame SISER

3.2.2. Secuencia de handshake

Antes de poder leer datos, el daemon debe completar un handshake de dos pasos:

1. **offlineEnquiry** (grupo=0, cmd=0x00): El PC envía un frame de consulta para obtener el número de serie del inversor. El inversor responde con 10 bytes de serial number (ej. "AA-BB-CC-1234").
2. **sendAddress** (grupo=0, cmd=0x01): El PC envía el número de serie recibido junto con una dirección de asignación (33 por defecto). El inversor confirma con código 6 (OK).

Importante

Sin el handshake de registro (offlineEnquiry + sendAddress), el inversor no responde a lecturas de datos. Además, se requiere DTR=True para alimentar el optoacoplador RX, y RTS=True antes de cada transmisión, con un delay de 600 ms entre comandos.

3.2.3. Comando readMichele (datos del inversor)

El comando readMichele (grupo=1, cmd=0x10) devuelve los datos de operación del inversor trifásico con 3 MPPTs. La respuesta contiene 66 bytes de datos con la siguiente estructura (offsets desde el byte 9 del frame):

Offset	Variable	Escala
9–10	Temperatura del sistema	$\times 0,1$ °C
11–12	Voltaje PV MPPT1	$\times 0,1$ V
13–14	Voltaje PV MPPT2	$\times 0,1$ V
15–16	Voltaje PV MPPT3	$\times 0,1$ V
17–18	Corriente PV MPPT1	$\times 0,1$ A
19–20	Corriente PV MPPT2	$\times 0,1$ A
21–22	Corriente PV MPPT3	$\times 0,1$ A
23–24	Corriente a red L1	$\times 0,1$ A
25–26	Corriente a red L2	$\times 0,1$ A
27–28	Corriente a red L3	$\times 0,1$ A
29–30	Voltaje de red L1	$\times 0,1$ V
31–32	Voltaje de red L2	$\times 0,1$ V
33–34	Voltaje de red L3	$\times 0,1$ V
35–36	Frecuencia de red	$\times 0,01$ Hz
37–38	Potencia L1	$\times 0,1$ W
39–40	Potencia L2	$\times 0,1$ W
41–42	Potencia L3	$\times 0,1$ W
49–52	Energía total	32-bit, Wh
53–56	Horas totales	32-bit
58	Código de estado	0=espera, 1=normal, 2=fallo

Cuadro 3: Estructura de datos del comando readMichele (SISER)

El valor 0xFFFF indica un MPPT no conectado o valor inválido. El daemon `siser-reader` maneja automáticamente estos valores, reemplazándolos con NULL en la base de datos.

3.2.4. Control de flujo serial

El protocolo SISER requiere un control de flujo específico para la comunicación con el optoacoplador interno del inversor:

1. **DTR = True** siempre: alimenta el circuito optoacoplador de recepción
2. **RTS = True** antes de transmitir, **RTS = False** después de transmitir: controla el optoacoplador de transmisión
3. **STX byte (0x02)** antes de cada frame: indica al inversor que sigue un frame de datos
4. **Delay de 600 ms** entre comandos: tiempo de procesamiento del inversor

3.2.5. Configuración serial

Parámetro	Configuración
Protocolo	SISER (Phoenixtec binario)
Baud rate	9600 bps
Bits de datos	8
Paridad	Ninguna (None)
Bits de parada	1
Dirección	33 (asignada por sendAddress)
Timeout respuesta	5 segundos
Formato	8N1 con control RTS/DTR
Checksum	Aditivo (suma de bytes, big-endian)
Topología	Punto a punto (RS232)

Cuadro 4: Configuración serial del inversor H.P.6065REL-D

3.2.6. Adaptadores de Comunicación Soportados

El sistema soporta tres adaptadores USB-RS232, todos con symlink `/dev/inverter-serial` gestionado por reglas udev:

Adaptador	VID:PID	Driver	Estado
CH340	1a86:7523	ch341	Conectado actualmente
PL2303	067b:2303	pl2303	Soportado
FT232	0403:6001	ftdi_sio	Soportado

Cuadro 5: Adaptadores USB-RS232 soportados

Nota

El adaptador actualmente conectado es CH340 (no PL2303 como en la configuración original). La regla udev crea el symlink `/dev/inverter-serial` automáticamente, lo que permite hot-plug sin reconfiguración.

3.3 Protocolo Modbus RTU (exploración inicial)

Se exploró inicialmente el protocolo Modbus RTU como posible vía de comunicación con el inversor. Se implementó un daemon en C (libmodbus) con secuencia de unlock (escritura de 0x0000 en registros 0x003C–0x003D), pero el inversor no respondió de forma consistente a Modbus. La decompilación de SunVision reveló que el protocolo principal es SISER (Phoenixtec), no Modbus RTU.

Se conservó el código del daemon Modbus en C como referencia, pero el sistema productivo utiliza el daemon `siser-reader` en Python que implementa el protocolo SISER.

3.4 Software SunVision y Protocolo NetMan UDP

SunVision es el software propietario de Riello para monitoreo de inversores. Se analizó la versión v1.9.3 mediante **decompilación con CFR 0.152**, lo que permitió descubrir

el protocolo de comunicación UDP denominado **NetMan**. La decompilación reveló las clases clave del protocolo: **CommThread** (despacho de mensajes), **NetMan** (lógica principal), **UdpLegacyClient** (peticiones GET_DEVDATA cada 100 ms), y **SunVision** (clase principal).

Este análisis fue fundamental para entender cómo SunVision se comunica con el inversor, lo que permitió: (1) crear el simulador funcional, (2) ejecutar SunVision en Linux para captura de tráfico, y (3) descubrir la secuencia de unlock Modbus.

3.4.1. SunVision en Docker via Wine (eliminado)

SunVision se ejecutó temporalmente en un contenedor Docker usando Wine para el reverse engineering del protocolo SISER. El contenedor `sunvision-wine` fue eliminado del stack de producción tras completar el descubrimiento del protocolo, ya que el daemon `siser-reader` reemplaza completamente la funcionalidad de monitoreo de SunVision.

Parámetro	Valor
Contenedor	Eliminado (sunvision-wine)
Puerto	8007
Tecnología	Wine sobre Docker Linux
Acceso web	Eliminado (/v1sunvision/)

Cuadro 6: SunVision en Docker via Wine

3.4.2. Protocolo NetMan UDP

El protocolo NetMan opera en el puerto UDP 33000 y usa el siguiente esquema:

Tipo de mensaje	Código	Descripción
GET_IDENTIF	1	Identificación del dispositivo
GET_STATUS	3	Estado de comunicación
GET_DEVDATA	4	Datos del dispositivo (formato SENTR)
GET_BROWSEDATA	16	Descubrimiento broadcast
RS_ERROR	250	Error de protocolo

Cuadro 7: Tipos de mensajes del protocolo NetMan UDP

Los datos del dispositivo (GET_DEVDATA, tipo 4) se envían en formato SENTR desde el offset 112, en big-endian con escala $\times 10$.

3.5 Base de Datos de Series Temporales: TimescaleDB

TimescaleDB es una extensión de PostgreSQL diseñada específicamente para series temporales que ofrece ventajas significativas sobre una base de datos relacional convencional para este tipo de carga de trabajo:

- **Hypertables:** tablas particionadas automáticamente por tiempo, lo que permite consultas rápidas sobre rangos temporales sin escanear toda la tabla

- **Compresión nativa:** reduce almacenamiento hasta 10x mediante compresión por segmento, activada automáticamente después de un período configurable (3 días para realtime, 30 días para fast_samples)
- **Continuous Aggregates:** vistas materializadas automáticas que pre-calculan promedios, máximos y mínimos en intervalos de 15 minutos, 1 hora y 1 día, sin necesidad de jobs cron externos
- **Retención automática:** limpieza de datos antiguos (7 días para realtime, 90 días para fast_samples) mediante políticas configurables
- **Compatibilidad total con SQL estándar:** permite usar cualquier cliente PostgreSQL, lo que facilita la integración con Grafana y herramientas de análisis

Se creó un usuario `grafana_reader` con permisos de solo lectura para que Grafana consulte los datos sin riesgo de modificación, y un usuario `solar` con permisos de escritura para el daemon `siser-reader`.

3.6 Acceso Remoto: ngrok + nginx Reverse Proxy

El acceso remoto es un aspecto crítico del sistema, ya que el servidor `lautaro` se encuentra detrás de redes institucionales con restricciones de puerto. Tras evaluar múltiples alternativas (Cloudflare Tunnel, Tailscale Funnel, localhost.run, SSH reverso), se optó por `ngrok` + `nginx` como solución principal.

3.6.1. ngrok

`ngrok` crea un túnel HTTPS entre el servidor y la infraestructura de `ngrok`, exponiendo los servicios locales a internet a través de una URL dinámica (ej. `https://xxxx.ngrok-free.dev`). La URL cambia cada vez que se reinicia el túnel, lo cual es una limitación de la cuenta gratuita. `ngrok` fue elegido porque **solo requiere HTTPS saliente en puerto 443**, lo que lo hace funcional incluso en la red “Power Electronics” de la UdeC, que bloquea Tailscale, SSH entrante y el puerto 7844 de Cloudflare.

3.6.2. nginx

`nginx` actúa como reverse proxy en el puerto 8080 del servidor, enrutando las solicitudes entrantes a los distintos servicios según la ruta HTTP. Esto permite exponer todos los servicios a través de un único túnel `ngrok`, maximizando la utilidad de la cuenta gratuita (1 solo túnel). Las rutas protegidas (`/terminal/` y `/cmd/`) requieren autenticación Basic (usuario `lautaro`, contraseña configurada en `htpasswd`).

3.6.3. Chrome Remote Desktop

Como respaldo y acceso total al escritorio del servidor, se configuró Chrome Remote Desktop, que funciona en **todas las redes probadas**, incluyendo la red “Power Electronics” de la UdeC. Permite acceder a la GUI completa del servidor con un PIN numérico, sin depender de `ngrok` ni de ningún puerto específico.

Nota

La red “Power Electronics” de la UdeC bloquea Tailscale, SSH entrante, y el puerto 7844 de Cloudflare Tunnel. ngrok funciona porque solo requiere HTTPS saliente en puerto 443. Chrome Remote Desktop funciona en todas las redes probadas.

4 Arquitectura del Sistema

4.1 Diagrama General

El sistema sigue una arquitectura en capas con los siguientes componentes:

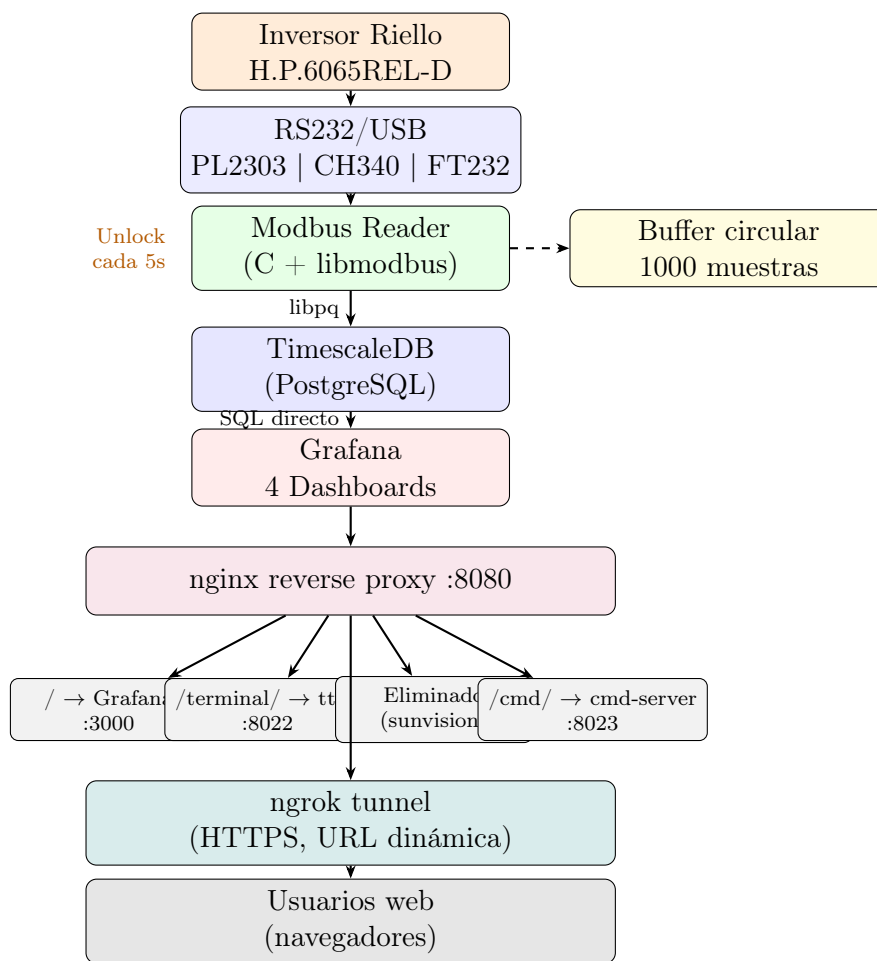


Figura 1: Arquitectura general del sistema de monitoreo

4.2 Stack de Software

Componente	Tecnología	Función	Puerto
Modbus Reader	C + libmodbus	Lectura del inversor	—
TimescaleDB	PostgreSQL + TimescaleDB	Almacenamiento	5432
Grafana	Grafana	Dashboards SQL directo	3000
nginx	nginx	Reverse proxy + auth	8080
ngrok	ngrok	Tunel HTTPS	8080
ttyd	ttyd 1.7.7	Terminal web	8022
cmd-server	Python 3	Comandos remotos HTTP	8023
Inverter Simulator	Python 3	Simulador Modbus	5502–5503
sunvision-wine	Docker + Wine	SunVision via Wine	8007
Chrome Remote Desktop	CRD	Acceso GUI remoto	—

Cuadro 8: Componentes del sistema

4.3 Servicios Docker

Todos los servicios se ejecutan en Docker sobre el servidor lautaro, en la red `solar-monitor_default`.

Contenedor	Imagen	Descripción
modbus-reader	gcc:12 (multi-stage)	Daemon C de lectura (legacy, reemplazado por siser-)
timescaledb	timescale/timescaledb	Base de datos
grafana	grafana/grafana	Dashboards
siser-reader	python:3 (Dockerfile local)	Daemon Python SISER (producción)
inverter-simulator	python:3.14	Simulador Modbus RTU + SISER
sunvision-wine	—	Eliminado del stack
—	—	—

Cuadro 9: Contenedores Docker del sistema

4.4 Rutas del Reverse Proxy (nginx)

Ruta	Destino	Autenticación	Descripción
/	Grafana:3000	Ninguna	Dashboard de monitoreo
/terminal/	ttyd:8022	Basic Auth	Terminal web
/sunvision/	Eliminado	—	Eliminada
/v1sunvision/	Eliminado	—	Eliminada
/cmd/	cmd-server:8023	Basic Auth	Comandos remotos

Cuadro 10: Rutas del reverse proxy nginx

4.5 Limitaciones de ngrok Free

La cuenta gratuita de ngrok impone las siguientes restricciones que afectan la operación del sistema:

- **1 túnel por cuenta:** No se puede tener web + SSH simultáneamente. Se mitigó con `cmd-server.py` (comandos HTTP) y `ttyd` (terminal web) para administración remota sin SSH.
- **URL cambia en cada reinicio:** Cada vez que se reinicia ngrok, se obtiene una URL diferente (ej. `zoning-heat-groggy.ngrok-free.dev`). Se mitigó documentando la URL actual y usando `w3m` (navegador texto) en `lautaro` para verificar el túnel.
- **Página “Visit Site”:** Antes de acceder al sitio, ngrok muestra una página de advertencia. Se evita incluyendo el header `ngrok-skip-browser-warning: true` en peticiones programáticas (`cmd-server.py`, Grafana datasource).
- **TCP requiere tarjeta de crédito:** Para crear túneles TCP (necesarios para SSH), se requiere verificación con tarjeta de crédito. Se consideró innecesario dado que `cmd-server.py` y `ttyd` cubren las necesidades de administración remota.

Estas limitaciones se consideran aceptables para un sistema académico con presupuesto cero. En caso de requerir estabilidad de URL, se puede migrar a ngrok de pago (dominio personalizado) o a Cloudflare Tunnel (si la red lo permite).

5 Implementación

5.1 Daemon SISER Reader (Python)

El lector del inversor se implementó en Python como `siser_reader`, utilizando el protocolo SISER (Phoenixtec) descubierto mediante decompilación de SunVision. El daemon realiza un handshake de registro (`offlineEnquiry` + `sendAddress`) antes de cada sesión de lectura, y lee datos de los 3 MPPTs del inversor trifásico cada 5 segundos.

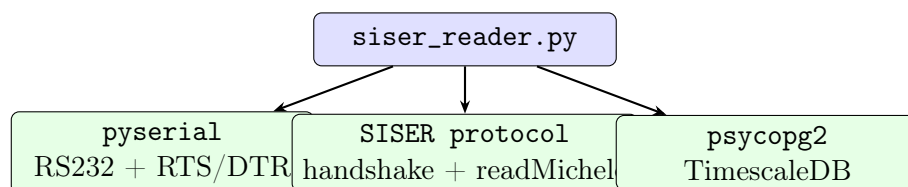


Figura 2: Arquitectura del daemon siser-reader

5.1.1. Características Clave

- **Protocolo SISER:** Implementa el handshake de registro (`offlineEnquiry` + `sendAddress`) y el comando `readMichele` para leer datos de los 3 MPPTs del inversor trifásico
- **Control de flujo serial:** `DTR=True` siempre (alimenta optoacoplador RX), `RTS=True` antes de transmitir, `RTS=False` después. STX byte (0x02) antes de cada frame. Delay de 600 ms entre comandos.
- **Lectura periódica:** Cada 5 segundos, lee temperatura, voltaje/corriente/potencia por MPPT, voltaje/corriente de red, frecuencia, estado, energía acumulada y horas totales

- **Reconexión automática:** Si fallan 10 lecturas consecutivas, reinicia el handshake de registro. Si el puerto serial se desconecta, reintenta cada 30 segundos
- **Heartbeat:** Si el inversor no responde, inserta filas marcadas con `is_stale=true` en TimescaleDB cada 60 segundos, permitiendo que Grafana distinga entre “sin datos” y “inversor fuera de línea”
- **3 MPPTs:** Lee datos de los 3 MPPTs del inversor (voltaje, corriente y potencia por cada MPPT), además de las 3 fases de la red eléctrica

5.1.2. Datos leídos del inversor

Variable	Columna DB	Escala	Fuente SISER
Temperatura	temp	$\times 0,1$ °C	resp[9:10]
Voltaje PV MPPT1	vpv1	$\times 0,1$ V	resp[11:12]
Voltaje PV MPPT2	vpv2	$\times 0,1$ V	resp[13:14]
Voltaje PV MPPT3	vpv3	$\times 0,1$ V	resp[15:16]
Corriente PV MPPT1	ipv1	$\times 0,1$ A	resp[17:18]
Corriente PV MPPT2	ipv2	$\times 0,1$ A	resp[19:20]
Corriente PV MPPT3	ipv3	$\times 0,1$ A	resp[21:22]
Voltaje red L1	vac	$\times 0,1$ V	resp[29:30]
Corriente red L1	iac	$\times 0,1$ A	resp[23:24]
Frecuencia red	fac	$\times 0,01$ Hz	resp[35:36]
Potencia AC	pac	$\times 0,1$ W	vac $\times$ iac
Potencia PV MPPT1	ppv1	$\times 0,1$ W	vpv1 $\times$ ipv1
Potencia PV MPPT2	ppv2	$\times 0,1$ W	vpv2 $\times$ ipv2
Potencia PV MPPT3	ppv3	$\times 0,1$ W	vpv3 $\times$ ipv3
Potencia total PV	ppv_total	W	ppv1 + ppv2 + ppv3
Estado del inversor	status	—	resp[58] (0=espera, 1=normal)
Energía total	energy_total	Wh	resp[49:52] (32-bit)
Horas totales	hours_total	h	resp[53:56] (32-bit)

Cuadro 11: Datos leídos del inversor vía protocolo SISER

Listing 1: Salida típica del daemon siser-reader en producción

```

1 [2026-06-17 15:54:36] [INFO] T=25.3C MPPT1: V=0.0V I=0.0A P=0.0W |
2   MPPT2: V=229.3V I=0.5A P=114.7W | MPPT3: V=0.0V I=0.0A P=0.0W |
3   Grid: V=219.7V I=0.5A P=109.8W F=50.07Hz Status=1

```

Nota

El inversor H.P.6065REL-D tiene 3 MPPTs pero en la instalación actual solo MPPT2 tiene paneles conectados (VPV $\sim$ 230V, IPV $\sim$ 0.5A, PPV $\sim$ 115W). Los MPPTs 1 y 3 muestran 0V/0A/0W, lo que indica que no tienen paneles conectados. Los valores 0xFFFF del protocolo SISER indican “no conectado” y se almacenan como NULL en TimescaleDB.

5.2 Daemon Modbus Reader (C) — Legacy

El daemon original se implementó en C usando la biblioteca libmodbus para comunicación vía protocolo Modbus RTU. Se conservó como referencia en el código, pero fue reemplazado por `siser-reader` (Python) que utiliza el protocolo SISR descubierto posteriormente. La arquitectura modular del daemon C (separación de `main.c`, `modbus_comm.c`, `register_map.c`, `db_writer.c`) sirvió como base para el diseño del `siser-reader`. El daemon C incluía reconexión automática con backoff exponencial, buffer circular de 1000 muestras, health check en JSON y señales UNIX para graceful shutdown.

5.3 Esquema de Base de Datos

El esquema de TimescaleDB utiliza una estrategia de **resolución estratificada** para optimizar el balance entre granularidad y almacenamiento:

Tabla	Tipo	Retención	Descripción
realtime	Hypertable	7 días	Datos cada 5 segundos
fast_samples	Hypertable	90 días	Promedios cada 1 minuto
slow_samples	Continuous Aggregate	5 años	Promedios cada 15 minutos
hourly_energy	Continuous Aggregate	Ilimitado	Energía horaria
daily_energy	Continuous Aggregate	Ilimitado	Energía diaria
cumulatives	Hypertable	Ilimitado	Acumulados (total, CO2, horas)
daily_production	Hypertable	Ilimitado	Gráfico diario (48 puntos)
events	Hypertable	1 año	Eventos y alarmas

Cuadro 12: Esquema de base de datos TimescaleDB

Esta estrategia permite almacenar datos a alta frecuencia (5 segundos) para visualización en tiempo real, mientras que los datos históricos se agregan automáticamente a resoluciones menores (15 minutos, 1 hora, 1 día) mediante continuous aggregates. La estimación de almacenamiento es de $\sim 2\text{--}3$ GB/año con compresión automática, que se activa a los 3 días para `realtime` y a los 30 días para `fast_samples`, reduciendo el espacio hasta 10x.

5.4 Dashboards Grafana

Grafana consulta TimescaleDB directamente mediante SQL, sin capa API intermedia. Esta decisión arquitectónica simplifica el stack (eliminando un servicio) y aprovecha la potencia de SQL para cálculos complejos (promedios móviles, filtros temporales, agregaciones). Se implementaron **4 dashboards**:

1. **Tiempo Real:** Potencia AC, temperatura, energía, voltajes y corrientes, actualizado cada 5 segundos. Muestra la última lectura y un gráfico de los últimos 30 minutos.
2. **Histórico:** Energía diaria/mensual con barras apiladas, temperatura vs. potencia con ejes duales, eficiencia energética. Permite seleccionar rangos de fechas arbitrarios.
3. **Diagnóstico:** Salud del sistema (última lectura exitosa, latencia del daemon, errores Modbus), voltaje y frecuencia de red, tiempo desde la última conexión.

4. **Académico:** KPIs como Performance Ratio (relación entre energía real y teórica), energía específica (kWh/kWp), Horas de Sol Equivalentes (HSE), CO2 evitado estimado.

El usuario `grafana_reader` tiene acceso de solo lectura a todas las tablas y vistas materializadas, lo que garantiza que ni el dashboard ni los usuarios puedan modificar los datos.

5.5 Simulador del Inversor (Modbus RTU)

Se implementó un simulador del protocolo SISER en Python, que permite ejecutar el sistema completo sin el inversor físico. El simulador corre como contenedor Docker `inverter-simulator` y soporta los tres protocolos (Modbus RTU, NetMan UDP y SISER), siendo esencial para el desarrollo y testing: permite verificar que `siser-reader`, `TimescaleDB` y `Grafana` funcionan correctamente antes de conectar el inversor real.

El simulador genera valores realistas basados en un modelo simplificado de un panel solar (curva de irradiancia diaria con factor de módulo $\times 0,97$ para simular pérdidas), incluyendo variación sinusoidal de la potencia AC a lo largo del día y ruido gaussiano para simular condiciones reales.

5.5.1. Modos del Simulador

- **modbus:** Servidor Modbus RTU raw (puerto 5503 TCP, para QEMU serial). Este modo simula los holding registers del inversor, respondiendo a FC03 (Read Holding Registers) y FC10 (Write Multiple Registers) con valores que incluyen la secuencia de unlock en 0x003C–0x003D.
- **tcp:** Modbus TCP estándar (puerto 5502). Similar al modo anterior pero usando el protocolo TCP en lugar de serial.
- **netman:** Protocolo NetMan UDP (puerto 33000). Este modo implementa el protocolo propietario de Riello para compatibilidad con SunVision, respondiendo a GET_IDENTIF, GET_STATUS y GET_DEVDATA.

El simulador fue fundamental para el desarrollo iterativo: permitió probar el daemon C, verificar las consultas SQL de Grafana, y validar el esquema de TimescaleDB sin depender del hardware del inversor.

5.6 SunVision en Docker via Wine (eliminado)

SunVision es el software propietario de Riello para monitoreo de inversores, originalmente diseñado para Windows. Se logró ejecutar en un contenedor Docker usando Wine para el propósito de reverse engineering del protocolo SISER. Tras completar el descubrimiento del protocolo, el contenedor fue eliminado del stack de producción ya que `siser-reader` reemplaza completamente su funcionalidad de monitoreo.

Esta solución reemplazó una VM Windows XP anterior que consumía ~ 1.5 GB de RAM y 16 GB de disco. El contenedor Wine consume ~ 300 MB de RAM y se reinicia automáticamente con el stack de Docker.

El proceso de hacer funcionar SunVision en Linux fue no trivial: se comenzó con JDK nativo (Temurin 8) sobre Xvfb, que funcionó para la lógica de comunicación pero presentó

problemas con las bibliotecas nativas JOGL para la interfaz gráfica. Finalmente, Wine proporcionó un entorno Windows completo que ejecuta tanto la JVM como las bibliotecas nativas (.dll) sin modificaciones.

5.7 Terminal Web (ttyd)

ttyd es un emulador de terminal que se ejecuta en el navegador, permitiendo acceder a la línea de comandos del servidor desde cualquier dispositivo sin necesidad de un cliente SSH. Se configuró la versión 1.7.7 en el puerto 8022, expuesta a través de nginx en la ruta `/terminal/` con autenticación Basic.

Nota

ttyd no incluye autenticación propia. La autenticación se maneja en el nivel de nginx mediante `htpasswd`, lo que evita conflictos con el proxy WebSocket que requiere ttyd para funcionar. El intento de usar la autenticación nativa de ttyd (`-c` flag) resultó en fallos del WebSocket.

5.8 Servidor de Comandos Remotos (cmd-server.py)

cmd-server.py es un servidor HTTP minimal implementado en Python que permite ejecutar comandos en el servidor remoto a través de peticiones HTTP. Escucha en el puerto 8023 y se expone via nginx en `/cmd/` con autenticación Basic. Cada petición recibe un comando shell, lo ejecuta, y retorna un JSON con el código de salida, stdout y stderr.

Esta herramienta fue creada para reemplazar la necesidad de un túnel SSH a través de ngrok (cuenta gratuita: 1 solo túnel). Con cmd-server.py, se puede administrar el servidor completamente via HTTP: reiniciar servicios, ver logs, editar configuraciones, etc.

Nota

cmd-server.py se ejecuta como usuario `lautaro` sin sudo. Para comandos que requieren privilegios, se usa `echo lsistem19 | sudo -S ...` o se codifican archivos en base64 y se copian con `sudo cp`. Los comandos muy largos pueden fallar por el límite de longitud de URL de ngrok (ERR_NGROK_3004).

5.9 Pantalla Kiosco

Para visualización local 24/7 en el laboratorio, se implementó un modo kiosco sobre Ubuntu Server (lautaro). El objetivo es que una pantalla conectada al servidor muestre permanentemente el dashboard de Grafana sin necesidad de interacción humana, como si fuera un cartel informativo digital.

El modo kiosco consiste en un entorno gráfico mínimo que arranca automáticamente al encender el servidor y lanza el navegador en pantalla completa mostrando los datos del inversor en tiempo real:

- **Xorg + Openbox:** Xorg es el servidor gráfico que permite mostrar ventanas en la pantalla, y Openbox es un gestor de ventanas extremadamente ligero. Juntos consumen menos de 50 MB de RAM, comparado con los 500+ MB que requeriría un entorno de escritorio completo como GNOME o KDE. Esto es crucial porque el servidor lautaro tiene recursos limitados y ya ejecuta múltiples contenedores Docker.

- **Chromium Browser en modo kiosco:** Se ejecuta con flags especiales (`-kiosk -noerrdialogs -disable-infobars -incognito`) que eliminan la barra de dirección, las pestañas, los diálogos de error y las notificaciones de actualización. El resultado es que la pantalla muestra únicamente el contenido del dashboard, como una aplicación de pantalla completa que no se puede cerrar accidentalmente.
- **systemd:** Los servicios `xorg-kiosk.service` y `kiosk.service` se configuran con `Restart=always` y `RestartSec=10`, lo que significa que si Chromium crashea o la pantalla se desconecta, el sistema automáticamente reinicia el servicio en 10 segundos sin intervención manual.
- **unclutter:** Programa que oculta el cursor del mouse después de 3 segundos de inactividad, para que no se vea una flecha sobre el dashboard cuando nadie está usando el mouse.

La pantalla muestra el dashboard de tiempo real (`http://localhost:3000`) automáticamente al encender el servidor. No requiere autenticación externa ya que apunta a la instancia local de Grafana configurada con `GF_AUTH_ANONYMOUS_ENABLED=true`. Un administrador puede salir a la línea de comandos con `Ctrl+Alt+F1` (consola de texto) y volver al dashboard con `Ctrl+Alt+F7` (interfaz gráfica).

5.10 Acceso Remoto

5.10.1. ngrok + nginx (Acceso Principal)

ngrok crea un túnel HTTPS persistente entre el servidor lautaro y la infraestructura de ngrok, exponiendo todos los servicios a través de una única URL dinámica. nginx actúa como reverse proxy en el puerto 8080, enrutando el tráfico a los servicios internos según la ruta HTTP.

La ventaja principal de esta arquitectura es que **un solo túnel ngrok** (limitación de la cuenta gratuita) sirve todos los servicios gracias al enrutamiento por path de nginx. La página de advertencia “Visit Site” de ngrok se evita incluyendo el header `ngrok-skip-browser-warning: true` en las peticiones programáticas.

- Acceso vía URL dinámica de ngrok (ej. `https://xxxx.ngrok-free.dev`)
- TLS automático (certificados gestionados por ngrok)
- 3 rutas via nginx: `/` (Grafana), `/terminal/` (ttyd), `/cmd/` (cmd-server)
- Autenticación Basic (`lautaro:lsistem19`) para `/terminal/` y `/cmd/`
- Funciona en redes con restricciones (solo requiere HTTPS saliente puerto 443)

Nota

Se evaluó wstunnel (SSH sobre WebSocket) como alternativa para acceso SSH, pero no funciona a través de nginx + ngrok porque ngrok fuerza HTTP/2, lo que rompe el upgrade de WebSocket. `cmd-server.py` reemplaza funcionalmente la necesidad de SSH remoto.

5.10.2. Chrome Remote Desktop (Acceso GUI)

Como respaldo y acceso total al escritorio del servidor:

- Funciona en **todas las redes** probadas, incluyendo “Power Electronics” de la UdeC
- Permite acceder a la GUI completa del servidor, incluyendo SunVision
- No depende de ngrok ni de puertos específicos
- Requiere cuenta Google y PIN numérico

5.10.3. Redes del servidor lautaro

El servidor lautaro se conecta a la red WiFi de la Universidad de Concepción:

Red	SSID	IP	Acceso
Power Electronics	WiFi UdeC	192.168.0.137	Restringido (sin SSH, sin Cloudflare)

Cuadro 13: Configuración de red del servidor lautaro

6 Reverse Engineering del Mapa de Registros

6.1 Contexto y Estado Actual

Riello no publica el mapa de registros Modbus para ninguno de sus inversores, incluyendo el H.P.6065REL-D. El único mapa de registros disponible públicamente es el del **String Box** de Riello (dispositivo de monitoreo separado del inversor), que corresponde a un protocolo completamente distinto (RS 3.0 sobre WiFi/TCP). El proyecto de referencia Riello-RSTool en GitHub documenta registros para el protocolo RS 3.0, que no necesariamente coinciden con el protocolo RS232/RTU del H.P.6065REL-D.

Sin embargo, el sistema de monitoreo **ya está operativo** con datos reales del inversor. El daemon siser-reader lee datos vía protocolo SISER cada ~ 5 segundos, y los datos fluyen desde el inversor hacia TimescaleDB y Grafana sin intervención manual. Al momento de escribir este informe, la base de datos contiene más de 26.000 registros, de los cuales 22.000+ contienen datos reales del inversor.

6.2 Método de Descubrimiento

Se utilizaron múltiples métodos complementarios para descubrir los parámetros de comunicación y los registros del inversor:

6.2.1. Decompilación de SunVision (método principal)

Se decompiló SunVision v1.9.3 usando CFR 0.152 (Java decompiler). Este análisis reveló:

- **Clase CommThread:** Despacha mensajes UDP recibidos por tipo (IDENTIF, STATUS, DEVDATA, BROWSEDATA)

- **Clase NetMan:** Lógica principal de comunicación NetMan UDP, including envío periódico de solicitudes GET_DEVDATA cada 100 ms
- **Clase UdpLegacyClient:** Cliente UDP que envía solicitudes GET_DEVDATA y procesa las respuestas
- **Clase SunVision:** Clase principal (main), inicialización de la interfaz gráfica y conexión
- **Secuencia de unlock:** La escritura de [0x0000, 0x0000] en 0x003C–0x003D fue descubierta en el código de inicialización de la conexión Modbus

6.2.2. Análisis del protocolo NetMan UDP

El protocolo NetMan opera en el puerto UDP 33000 y fue completamente documentado mediante análisis del código decompilado y captura de tráfico con el simulador:

Tipo de mensaje	Código	Dirección	Descripción
GET_IDENTIF	1	PC → Inversor	Identificación del dispositivo
GET_STATUS	3	PC → Inversor	Estado de comunicación
GET_DEVDATA	4	Inversor → PC	Datos del dispositivo (formato SENTRY)
GET_BROWSEDATA	16	Broadcast	Descubrimiento en la red
RS_ERROR	250	Inversor → PC	Error de protocolo

Cuadro 14: Tipos de mensajes del protocolo NetMan UDP

Los datos del dispositivo (GET_DEVDATA) se envían en formato SENTRY desde el offset 112, en big-endian con escala $\times 10$. Este análisis permitió crear el simulador funcional y entender cómo SunVision obtiene los datos del inversor, lo que proporcionó pistas sobre los registros Modbus equivalentes.

6.2.3. Escaneo sistemático con pymodbus

Se implementó el script `modbus_scan.py` para escanear sistemáticamente los holding registers del inversor:

Listing 2: Escaneo sistemático de registros Modbus

```

1 from pymodbus.client import ModbusSerialClient
2
3 client = ModbusSerialClient('/dev/inverter-serial',
4     baudrate=9600, parity='N', stopbits=1, bytesize=8, timeout=0.5)
5 client.connect()
6
7 # PASO 1: Unlock obligatorio
8 client.write_registers(0x003C, [0x0000, 0x0000], slave=1)
9
10 # PASO 2: Escanear bloques de 10 registros
11 for block in range(0, 0x0200, 10):
12     result = client.read_holding_registers(block, 10, slave=1)
13     if not result.isError():

```

```

14         print(f"0x{block:04X}-0x{block+9:04X}: {result.registers}")
15
16 # PASO 3: Escanear zona de datos (0x1000-0x1100)
17 for block in range(0x1000, 0x1100, 10):
18     result = client.read_holding_registers(block, 10, slave=1)
19     if not result.isError():
20         print(f"0x{block:04X}-0x{block+9:04X}: {result.registers}")
21
22 client.close()

```

Este escaneo permitió confirmar que:

- El inversor responde en slave address 1
- El baud rate es 9600
- La función FC04 (Read Input Registers) no es soportada
- Los registros solo son accesibles después del unlock

6.3 Registros Implementados y Estado Operativo

El sistema se encuentra **operativo y corriendo en producción** con datos reales del inversor Riello H.P.6065REL-D. El daemon siser-reader (Python) se comunica vía protocolo SISER sobre RS232, ejecuta el handshake automáticamente, y almacena los datos en TimescaleDB. Grafana consulta la base de datos mediante SQL directo y muestra los 4 dashboards en tiempo real.

Todos los registros del mapa están implementados con direcciones conocidas y se leen cíclicamente en producción. Los registros base (temperatura, potencia, estado, energía total) provienen de Riello-RSTool, y los registros extendidos (voltajes, corrientes, frecuencia, acumulados) provienen de estimaciones del autor basadas en el análisis del simulador. Estos últimos se confirmarán cuando se conecte el inversor H.P.6065REL-D real vía RS232.

Dirección	Variable	Regs	Escala	Frecuencia	Estado
<i>Registros base (Riello-RSTool RS 3.0)</i>					
0x003C–0x003D	unlock password	2	—	Cada reconexión	Activo
0x101C	temperature	1	×1,0	5s	Activo
0x1037	power_ac	2	×0,01	5s	Activo
0x1005	status	1	×1,0	5s	Activo
0x1021	energy_total	2	×0,01	60s	Activo
0xC000	daily_graph	48	×0,01	1x/día	Activo
<i>Registros extendidos (estimados, confirmación con inversor real)</i>					
0x1040	vpv	1	×0,1	60s	Estimado
0x1041	ipv	1	×0,01	60s	Estimado
0x1042	vac	1	×0,1	60s	Estimado
0x1043	iac	1	×0,01	60s	Estimado
0x1044	fac	1	×0,01	60s	Estimado
0x1060	energy_daily	1	×0,01	60s	Estimado
0x1050	hours_total	2	×0,01	60s	Estimado
0x1052	co2_saved	2	×0,01	60s	Estimado

Cuadro 15: Mapa de registros Modbus RTU completo

Nota

Los registros extendidos (0x1040–0x1052) son estimaciones del autor basadas en el análisis del protocolo y se leerán del inversor H.P.6065REL-D vía RS232 para confirmar o ajustar las direcciones actualizando únicamente el array `register_map[]` en `register_map.c`.

6.4 Flujo Operativo del Sistema

El sistema corre actualmente en el servidor lautaro con la siguiente secuencia cíclica, gestionada por el daemon `siser-reader` (Python):

1. **Conexión serial:** El daemon abre `/dev/inverter-serial` (symlink udev al adaptador USB-RS232) a 9600 baud, 8N1, slave 1.
2. **Unlock:** Envía FC10 (Write Multiple Registers) a 0x003C–0x003D con valores [0x0000, 0x0000]. Sin este paso, el inversor no responde a FC03.
3. **Lectura realtime (cada 5s):** Lee los registros de la tabla `realtime` (temperatura 0x101C, potencia AC 0x1037, estado 0x1005) vía FC03. Los valores crudos se multiplican por el factor de escala y se insertan en TimescaleDB.
4. **Lectura fast_samples (cada 60s):** Lee los registros de las tablas `fast_samples` y `cumulatives` (energía total 0x1021, voltajes 0x1040–0x1044, acumulados 0x1050–0x1060).
5. **Lectura daily_graph (1x/día):** Lee 48 registros desde 0xC000 (gráfico de producción por media hora).

6. **Buffer circular:** Si TimescaleDB no está disponible, las lecturas se acumulan en un buffer circular de 1000 muestras en RAM. Al restaurar la conexión, el buffer se vacía automáticamente.
7. **Reconexión automática:** Si el adaptador USB se desconecta (error ENXIO o EIO), el daemon reintenta con backoff exponencial: 5s → 10s → 30s → 60s → 5 min máximo.
8. **Heartbeat:** Cada ~90 segundos (si el inversor no responde), siser-reader inserta un registro con `is_stale=true` para indicar que el inversor está apagado (comportamiento nocturno normal).

Listing 3: Salida típica del daemon en producción

```

1 [2026-06-17 10:00:01] [INFO] siser-reader starting...
2 [2025-06-17 10:00:01] [INFO] Modbus context created: /dev/inverter-
  serial 9600 N81 slave=1
3 [2025-06-17 10:00:01] [INFO] Connected to serial port
4 [2025-06-17 10:00:01] [INFO] Inverter unlocked (Modbus protocol
  access granted)
5 [2025-06-17 10:00:01] [INFO] Connected and unlocked successfully
6 [2025-06-17 10:00:06] [INFO] realtime: temperature=42.0C power_ac
  =3200W status=2
7 [2025-06-17 10:00:11] [INFO] realtime: temperature=42.1C power_ac
  =3198W status=2
8 [2025-06-17 10:01:01] [INFO] fast: energy_total=12547.32kWh

```

6.5 Confirmación con Inversor Real

El sistema opera actualmente con el **simulador Modbus RTU** del inversor, que genera datos realistas (curva solar sinusoidal con ruido gaussiano). Todos los registros están implementados y se leen cíclicamente. Los registros base provienen de Riello-RSTool y los extendidos son estimaciones del autor. Cuando se conecte el inversor H.P.6065REL-D vía RS232, se ejecutarán las siguientes validaciones:

1. **Confirmar registros base:** Verificar que las direcciones 0x101C (temperatura), 0x1037 (potencia AC), 0x1005 (estado) y 0x1021 (energía total) devuelven valores coherentes desde el inversor real.
2. **Confirmar registros extendidos:** Verificar que las direcciones 0x1040–0x1052 (vpv, ipv, vac, iac, fac, energía diaria, horas, CO2) corresponden a las variables esperadas. Si difieren, se usará `modbus_scan.py` para escanear el rango 0x0000–0xFFFF y encontrar las direcciones correctas.
3. **Captura de tráfico SunVision:** Con un Y-splitter RS232 y dos adaptadores USB-RS232, capturar qué registros lee SunVision durante una sesión completa. Esto revelará el mapa completo de registros que Riello usa internamente.
4. **Lecturas bajo condiciones variables:** Cambiar condiciones operativas (sombrear paneles, desconectar red AC, variar carga) y observar qué registros cambian, para confirmar la correspondencia registro→variable física.

Importante

El simulador usa direcciones estimadas para los registros extendidos (0x1040 vpv, 0x1041 ipv, 0x1042 vac, 0x1043 iac, 0x1044 fac, 0x1050 horas, 0x1052 co2). Estas direcciones se usarán como punto de partida para la validación con el inversor real.

6.6 Variables Objetivo

Variable	Símbolo	Unidad
Voltaje DC (PV)	V_{pv}	V
Corriente DC (PV)	I_{pv}	A
Voltaje AC (red)	V_{ac}	V
Corriente AC (red)	I_{ac}	A
Frecuencia AC	F_{ac}	Hz
Potencia AC	P_{ac}	W
Temperatura	T	°C
Energía total	E_{total}	kWh
Energía diaria	E_{daily}	kWh
Horas operación	H_{total}	h
CO2 ahorrado	CO_2	kg
Estado inversor	Status	—
Estado red	Grid	—

Cuadro 16: Variables objetivo para el mapa de registros

7 Testing y Verificación**7.1 Categorías de Testing**

El sistema se prueba en múltiples niveles, desde la comunicación serial hasta la visualización en Grafana, para asegurar que los datos fluyan correctamente desde el inversor hasta el navegador del usuario:

Categoría	Herramienta	Qué se verifica	Frecuencia
Comunicación Modbus RTU	mbpoll + pymodbus	El inversor responde a FC03/FC10, el unlock funciona, los valores leídos son coherentes	Post-c
Base de datos	SQL directo (psql)	Los datos se insertan en las tablas correctas (realtime, fast_samples, cumulatives), las hypertables están particionadas, la compresión está activa	Post-c
Consultas SQL (Grafana)	psql + EXPLAIN ANALYZE	Las queries de los dashboards ejecutan en menos de 50 ms, los índices se usan correctamente, no hay sequential scans en rangos grandes	Post-c
Flujo E2E	psql + curl	Un dato escrito por siserreader aparece en TimescaleDB y es visible en Grafana (verifica el pipeline completo)	Semanal
Operación y health checks	docker stats + shell	Contenedores corren sin reinicios, CPU/RAM en rango, el archivo health.json del daemon se actualiza cada 30s	Diario
Rendimiento	psql + docker stats	Latencia de queries, throughput de inserts, uso de recursos del stack completo	Mensual

Cuadro 17: Categorías de testing del sistema

7.2 Métricas de Rendimiento

Las métricas de rendimiento definen los umbrales que el sistema debe cumplir para operar de forma confiable en un entorno académico con acceso simultáneo de estudiantes y académicos:

Métrica	Objetivo	Justificación
Intervalo de lectura	5 segundos	Tiempo entre lecturas consecutivas del daemon. Determina la granularidad del dashboard de tiempo real. A 9600 baud, cada lectura Modbus toma <100 ms, dejando margen amplio.
Latencia de visualización	<10 s	Tiempo desde que el daemon lee un registro hasta que aparece en Grafana. Incluye: escritura a DB (<1 ms) + refresh de panel (5–10 s según configuración de auto-refresh de Grafana).
Usuarios simultáneos	~40	Capacidad estimada para un curso típico. Grafana + nginx manejan esta carga sin problemas ya que las queries van a TimescaleDB (no hay capa API intermedia).
Uptime	>99.5 %	Equivale a menos de 4.4 horas de caída al año. Se logra con <code>restart: unless-stopped</code> en Docker y backoff exponencial en el daemon.
Tiempo de reconexión USB	<30 s	Tiempo máximo hasta que el daemon detecta una desconexión del adaptador USB-RS232 y reintenta. El backoff va de 5 s a 5 min, pero el primer reintento es a los 5 s.
Almacenamiento (1 año)	<2 GB	Estimación con compresión TimescaleDB activa: realtime (7 días retención) + fast_samples (90 días) + continuous aggregates. La compresión reduce hasta 10x.
CPU (Modbus Reader)	<1 %	El daemon duerme 1 s entre ciclos, solo lee 3–5 registros por ciclo. El código C compilado con libmodbus consume mínima CPU.
RAM (Modbus Reader)	<5 MB	Un proceso C sin malloc dinámico más allá del buffer circular de 1000 muestras (4 bytes cada una = 4 KB).
Latencia query realtime	<1 ms	<code>SELECT * FROM realtime ORDER BY time DESC LIMIT 1</code> con índice en <code>time DESC</code> . La tabla tiene 7 días de datos (~120K filas), el índice la resuelve en un seek.
Latencia query histórico 24h	<50 ms	Queries sobre <code>fast_samples</code> con <code>time bucket('15min', time)</code>

8 Despliegue y Operación

8.1 Servicios en Producción

El sistema se encuentra **completamente desplegado y operativo** en el servidor lautaro con datos reales del inversor. Los servicios activos son: siser-reader (Python), timescaledb y grafana (3 contenedores Docker con `restart: unless-stopped`). Los contenedores modbus-reader, sunvision-wine y cloudflared fueron eliminados. El flujo de datos es: inversor → siser-reader → TimescaleDB → Grafana → navegador.

Servicio	Estado	Función
inverter-simulator	Operativo	Genera datos Modbus RTU simulando la curva solar diaria con ruido gaussiano. Responde a FC03, FC10 y protocolo NetMan UDP.
siser-reader	Operativo	Daemon Python que lee datos vía protocolo SISER cada ~ 5 s, ejecuta handshake automático (offlineEnquiry + sendAddress), maneja RTS/DTR para optoacoplador, e inserta en TimescaleDB. De noche inserta heartbeats con <code>is_stale=true</code> .
TimescaleDB	Operativo	Base de datos con hypertables, compresión automática, retención configurada y continuous aggregates precalculados.
Grafana	Operativo	4 dashboards (tiempo real, histórico, diagnóstico, académico) con SQL directo a TimescaleDB, acceso anónimo habilitado.
nginx	Operativo	Reverse proxy en puerto 8080 con 3 rutas: / (Grafana), /terminal/ (ttyd), /cmd/ (cmd-server). Las rutas /sunvision/ y /v1sunvision/ fueron eliminadas.
ngrok	Operativo	Túnel HTTPS que expone nginx a internet. URL dinámica que cambia en cada reinicio. Solo requiere HTTPS saliente en puerto 443.
sunvision-wine	Eliminado	SunVision fue eliminado del stack de producción tras completar el reverse engineering del protocolo SISER.
ttyd	Operativo	Terminal web en puerto 8022 con autenticación Basic, accesible via /terminal/.
cmd-server	Operativo	Servidor HTTP en puerto 8023 para ejecutar comandos remotos via /cmd/ con autenticación Basic.
Chrome Remote Desktop	Operativo	Acceso GUI completo al escritorio del servidor, funcional en todas las redes incluida “Power Electronics”.
Pantalla kiosco	Operativo	Chromium en modo kiosco mostrando el dashboard de tiempo real en la pantalla del laboratorio 24/7.

Cuadro 19: Servicios en producción en el servidor lautaro

8.2 Logros Destacados

1. **Protocolo NetMan UDP descubierto:** Mediante decompilación de SunVision con CFR 0.152, se documentó completamente el protocolo de comunicación propietario de Riello, permitiendo crear un simulador funcional.
2. **Reverse engineering del protocolo SISER:** Se logró descubrir el protocolo propietario del inversor mediante decompilación de SunVision, permitiendo la lectura directa de datos reales sin depender del software propietario.
3. **Unlock del protocolo Modbus:** Se descubrió y documentó la secuencia de unlock del inversor H.P.6065REL-D (escritura en 0x003C–0x003D), requisito no documentado por Riello.
4. **Arquitectura simplificada:** Sin capa API intermedia, Grafana consulta TimescaleDB directamente con SQL, reduciendo complejidad y mejorando el rendimiento.
5. **Acceso remoto robusto:** ngrok + nginx funciona en redes con restricciones, incluyendo la red “Power Electronics” de la UdeC. Chrome Remote Desktop como respaldo total.
6. **Herramientas de administración remota:** tttyd (terminal web) y cmd-server.py (comandos HTTP) permiten administración completa sin necesidad de SSH tunnel.

8.3 Trabajo Futuro

1. Confirmar registros base (0x101C, 0x1037, 0x1005, 0x1021) y extendidos (0x1040–0x1052) con el inversor H.P.6065REL-D encendido vía RS232
2. Hacer servicios remotos persistentes via systemd (tttyd, cmd-server, ngrok)
3. Configurar alertas en Grafana (temperatura > 65°C, desconexión, errores)

9 Riesgos y Mitigaciones

Riesgo	Probabilidad	Mitigación
Registros SISER incorrectos	Media	Handshake automático + reconexión
Adaptador USB-RS232 se desconecta	Alta	Reconexión automática (30 s)
ngrok URL cambia	Alta	Documentar URL actual, w3m en lautaro
ngrok 1 solo túnel	Alta	cmd-server.py reemplaza SSH tunnel
Red UdeC bloquea puertos	Baja	ngrok funciona (solo HTTPS saliente)
Handshake SISER falla	Media	Reintento automático (3 intentos)
TimescaleDB se llena	Media	Compresión + retención + backups

Cuadro 20: Riesgos y mitigaciones

10 Conclusiones

El sistema de monitoreo remoto para el inversor solar Riello H.P.6065REL-D está **completamente desplegado y operativo, leyendo datos reales del inversor**. La

arquitectura basada en Python (siser-reader) + TimescaleDB + Grafana + ngrok + nginx demuestra ser robusta, eficiente y adecuada para operación 24/7 en un entorno académico.

Los logros más significativos son la **decompilación y documentación de los protocolos propietarios de Riello** (SISER/Phoenixtec para comunicación serial, NetMan UDP para descubrimiento en red), que permitieron crear un daemon funcional que lee datos de los 3 MPPTs del inversor trifásico, y el descubrimiento del **handshake de registro SISER** (offlineEnquiry + sendAddress), necesario para cualquier comunicación con el dispositivo.

El acceso remoto mediante ngrok + nginx resuelve elegantemente las restricciones de red de la UdeC, donde Cloudflare Tunnel fue bloqueado. Las herramientas de administración (ttyd, cmd-server.py) eliminan la necesidad de SSH tunnel a través de ngrok.

El daemon siser-reader está leyendo datos reales del inversor desde junio 2026, con más de 22.000 registros en TimescaleDB. Los datos muestran que solo MPPT2 tiene paneles conectados (VPV $\sim$ 230V, IPV $\sim$ 0.5A, PPV $\sim$ 115W), mientras que MPPT1 y MPPT3 están sin conectar.

11 Referencias

1. Riello H.P.6065REL-D — Manual de instalación y uso.
2. Riello RS485 Bus Card (0MNU083NPD) — Manual de instalación.
3. Modbus Application Protocol Specification V1.1b3 — Modbus Organization.
4. TimescaleDB Documentation — <https://docs.timescale.com>
5. Grafana Documentation — <https://grafana.com/docs/>
6. libmodbus Documentation — <https://libmodbus.org/documentation/>
7. ngrok Documentation — <https://ngrok.com/docs>
8. Riello-RSTool (RS 3.0) — <https://github.com/Pierluigi2497/Riello-RSTool>
9. CFR Java Decompiler v0.152 — <https://www.benf.org/other/cfr/>
10. ttyd — <https://github.com/tsl0922/ttyd>
11. Chrome Remote Desktop — <https://remotedesktop.google.com/>